PROJECT DOCUMENTATION

# mini-baas

## A Self-Adapting, Database-Agnostic Backend-as-a-Service

DATE March 06, 2026

STATUS Final Release

# Table of Contents

01 1. Key Concepts & Architecture

1.1 Metadata-Driven Architecture

1.2 Multi-Tenant Isolation Strategy

02 2. Technology Stack

03 3. Request Flow & Orchestration

04 4. Modular Directory Structure (Domain-Driven Design)

05 5. System Maturity Stages (Action Plan)

5.1 Stage 1: Logical Multi-Tenancy & Metadata Foundation

5.2 Stage 2: The Adapter Pattern & Basic Data Plane

5.3 Stage 3: Dynamic Validation

5.4 Stage 4: Advanced Query DSL & Relations

5.5 Stage 5: Hook Execution & Background Jobs

5.6 Stage 6: Enterprise API Gateway & Observability

5.7 Stage 7: Billing Engine & Schema Evolution

06 6. Research & Validation: The Power of Metadata

6.1 Alignment with our App Factory

6.2 Architectural Flow Diagram

07 7. Enterprise SaaS Capabilities: Scalability, Observability & Evolution

7.1 7.1 Scalability and API Gateway Multitenancy

7.2 7.2 Multi-Tenant Observability

7.3 7.3 Billing & Usage Metering (The Event-Driven Approach)

7.4 7.4 Schema Evolution & Metadata Versioning

# mini-baas — A Self-Adapting, Database-Agnostic Backend-as-a-Service

> **Core principle:** We are not building an app. We are building an App Factory.
> Our backend must transform itself at runtime to serve any business model it has never seen before — without a single line of hardcoded schema, controller, or model.

---


- 1. Key Concepts & Architecture
- Metadata-Driven Architecture
  - 📄 Example: The “Master Document” (The Tenant’s DNA)
  - 💻 Example: Control Plane Implementation (Mongoose)
- Multi-Tenant Isolation Strategy
- 2. Technology Stack
- 3. Request Flow & Orchestration
- 4. Modular Directory Structure (Domain-Driven Design)
- 5. System Maturity Stages (Action Plan)
- Stage 1: Logical Multi-Tenancy & Metadata Foundation
- Stage 2: The Adapter Pattern & Basic Data Plane
- Stage 3: Dynamic Validation
- Stage 4: Advanced Query DSL & Relations
- Stage 5: Hook Execution & Background Jobs
- Stage 6: Enterprise API Gateway & Observability
- Stage 7: Billing Engine & Schema Evolution
- 6. Research & Validation: The Power of Metadata
- Alignment with our App Factory
- Architectural Flow Diagram
- 7. Enterprise SaaS Capabilities: Scalability, Observability & Evolution

- 7.1 Scalability and API Gateway Multitenancy
- 7.2 Multi-Tenant Observability
- 7.3 Billing & Usage Metering (The Event-Driven Approach)
- 7.4 Schema Evolution & Metadata Versioning


---

# 1. Key Concepts & Architecture

## Metadata-Driven Architecture

Most backends are static: a developer writes a schema and deploys. This breaks completely for a platform like ours. **The backend has zero knowledge of any user's data model at build time.** Instead of hardcoded ORM models (like Prisma or TypeORM entities), users define their data model as JSON metadata. This "Master Document" is stored in our **System DB**. At request time, NestJS reads this metadata, builds a runtime validator, and generates a dynamic query builder on the fly. The backend *becomes* the right backend for each user, on every request.

### 📄 EXAMPLE: THE "MASTER DOCUMENT" (THE TENANT'S DNA)

This JSON document represents everything the Data Plane needs to know to serve a specific tenant. It replaces static code completely.

```
    {
      "_id": "64a7b...89c",
      "tenantId": "ws_123",
      "status": "active",
      "database": {
        "engine": "postgresql",
        "uri": "postgres://user:pass@db.example.com:5432/tenant_db"
      },
      "schema": {
        "books": {
          "fields": {
            "title": { "type": "string", "required": true },
            "price": { "type": "number", "default": 0 }
          }
        }
      },
      "hooks": {
        "books": {
          "beforeCreate": "function(data) { if(data.price < 0) throw new Error('Invalid
price'); return data; }"
        }
      },
      "permissions": {
        "books": { "read": ["public", "admin"], "create": ["admin"] }
      },
      "version": 1
    }
```

## 💻 EXAMPLE: CONTROL PLANE IMPLEMENTATION (MONGOOSE)

To store this flexible, schema-less structure in our System DB, we use MongoDB and Mongoose. Here is how the schema is defined in the Control Plane:

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document, Schema as MongooseSchema } from 'mongoose';

@Schema({ timestamps: true })
export class TenantMetadata extends Document {
  @Prop({ required: true, unique: true, index: true })
  tenantId: string; // e.g., 'ws_123'

  @Prop({ required: true })
  status: string; // 'active', 'suspended'

  @Prop({ type: MongooseSchema.Types.Mixed, required: true })
  database: { engine: 'postgresql' | 'mongodb' | 'mysql'; uri: string };

  @Prop({ type: MongooseSchema.Types.Mixed, default: {} })
  schema: Record<string, any>; // The dynamic dictionary of tables/collections

  @Prop({ type: MongooseSchema.Types.Mixed, default: {} })
  hooks: Record<string, any>; // JS code stored as text for isolated-vm

  @Prop({ type: MongooseSchema.Types.Mixed, default: {} })
  permissions: Record<string, any>; // RBAC/ABAC rules

  @Prop({ default: 1 })
  version: number;
}

export const TenantMetadataSchema = SchemaFactory.createForClass(TenantMetadata);
```

*(Reference: https://mongoosejs.com/docs/schematypes.html#mixed )*

## Multi-Tenant Isolation Strategy

To securely serve unbounded tenant diversity, the platform is strictly divided into two macro domains:

- **The Control Plane:** Responsible for governance. It manages tenant lifecycles, billing, policies, and stores the metadata (the schema definitions and configurations). Control plane failure must *not* break data plane execution.
- **The Data Plane:** The stateless, horizontally scalable runtime that executes user workloads. It caches metadata, validates incoming data, and injects the right database adapter per request.

**Isolation is enforced at three layers:**

1. **Data Isolation Strategy:** A hybrid model: Shared DB + Separate Schemas. Small tenants use a Shared DB with Row-Level Security (or tenant-ID scopes). Enterprise tenants get a dedicated database.

2. **Compute Isolation:** Strict resource quotas and the use of sandboxed environments for custom user code.
3. **Cache Isolation:** Every cache key in Redis is strictly namespaced (e.g., `tenant:ws_123:metadata`).

---

## 2. Technology Stack

To achieve this dynamic behavior, we selected a stack optimized for modularity, runtime flexibility, and isolation.

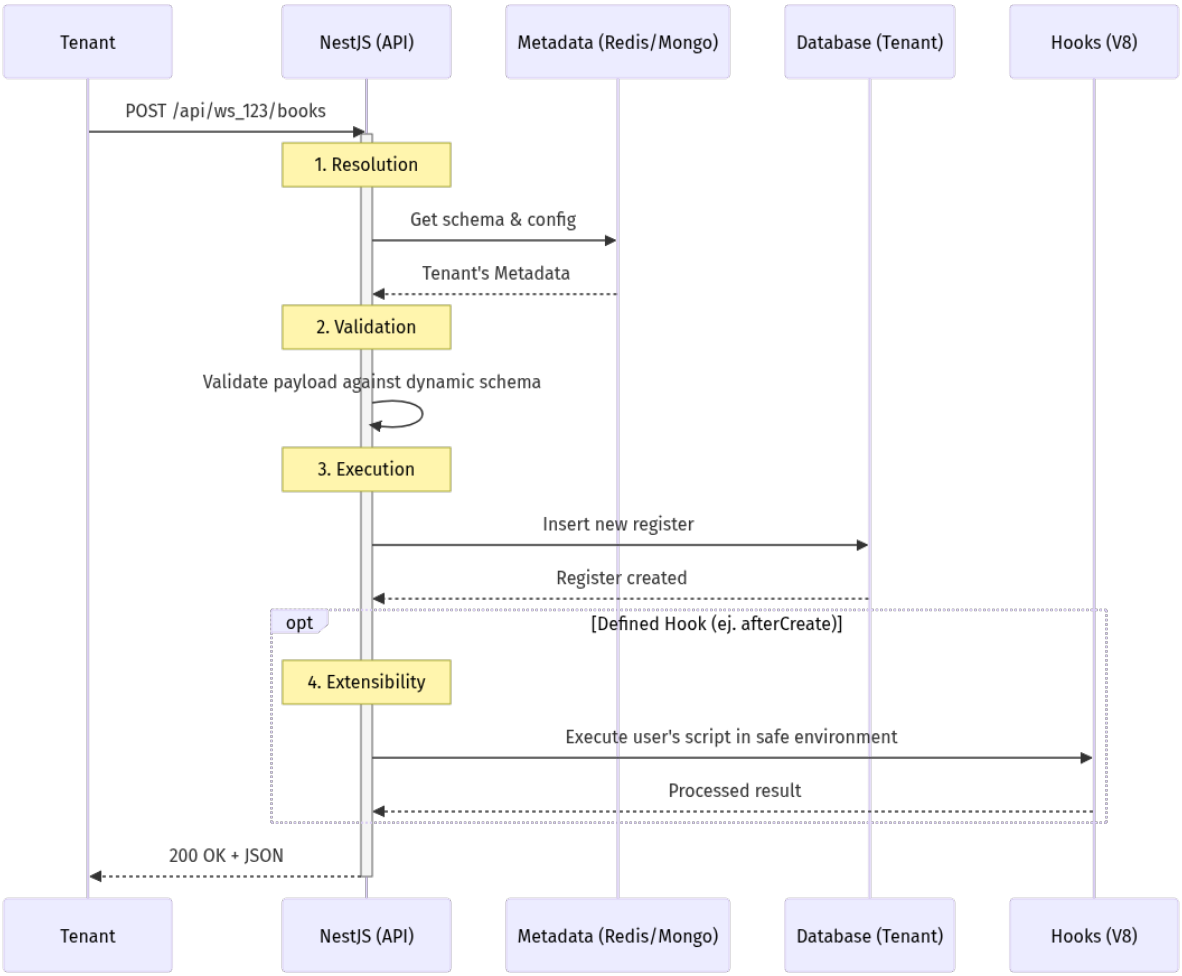
| Layer | Technology | Why it's the right choice | Docs |
|---|---|---|---|
| **Framework** | NestJS (TypeScript) | Its powerful Dependency Injection (DI) and Request-Scoped providers are essential for implementing the Adapter Pattern. It allows us to inject a different DB engine per request. | NestJS |
| **System DB** | MongoDB & Mongoose | Perfect for the Control Plane. Schemas (metadata) are inherently dynamic, nested JSON objects. A document database allows us to store and evolve tenant configurations without writing SQL migrations. | MongoDB |
| **SQL Engine** | Knex.js | A dynamic query builder. Since we don't have static models, Knex allows us to construct complex SQL queries (SELECT, JOIN) from abstract JSON syntax trees at runtime. | Knex.js |
| **NoSQL Engine** | MongoDB Native Driver | Provides direct, schema-free collection access for tenants bringing their own NoSQL databases. | Mongo Node |
| **Validation** | AJV / Zod | Since we lack TypeScript DTOs, these tools allow us to compile validation schemas directly from the stored JSON metadata at runtime to secure the Data Plane. | Zod |
| **Sandbox** | isolated-vm | Safely executes user-defined JavaScript hooks (onCreate, onUpdate). It creates a secure V8 isolate so untrusted user code cannot access or crash our Node.js environment. | isolated-vm |
| **Cache & Jobs** | Redis + BullMQ | Tenant-aware caching for metadata (crucial for performance) and async task queues for webhooks and background jobs. | BullMQ |

---

## 3. Request Flow & Orchestration

How do these technologies connect? Let's trace the lifecycle of a single request: `POST /api/ws_123/books`

1. **Intercept & Context (NestJS + Redis):** The `TenantInterceptor` reads the `ws_123` ID from the URL/Header. It checks **Redis** for the tenant's cached metadata. If missing, it fetches the "Master Document" from the **MongoDB System DB** and caches it.
2. **Adapter Injection (NestJS DI):** The `DatabaseProvider Factory` reads the tenant's config ( `dbType: "postgresql"` ). NestJS dynamically instantiates the `PostgresAdapter` with the tenant's connection string and injects it into the request scope.
3. **Dynamic Validation (AJV/Zod):** The `DynamicController` receives the generic payload. The `ValidationEngine` extracts the `books` schema from the metadata, compiles it into a Zod/AJV rule, and validates the incoming JSON body.
4. **Query Execution (Knex / Mongo Driver):** The `DynamicService` calls `this.db.create('books', validatedData)`. The `PostgresAdapter` uses **Knex.js** to translate this into an `INSERT INTO books...` SQL statement and executes it against the external tenant database.
5. **Hook Execution (isolated-vm):** If the metadata defines an `afterCreate` hook, the `HookRuntime` spins up a secure V8 isolate via **isolated-vm**, injects the newly created record, and executes the user's custom JavaScript function.
6. **Response:** The `TransformLayer` normalizes the result and returns a consistent JSON payload to the client.




## 4. Modular Directory Structure (Domain-Driven Design)

To maintain sanity and scalability, the codebase enforces strict boundaries. Engines know nothing about HTTP, and Controllers know nothing about SQL.

```
src/
├── main.ts                     # Application entry point
├── app.module.ts               # Root module assembling the App Factory
│
├── common/                     # Shared tools (Business-agnostic)
│   ├── interceptors/           # e.g., TenantInterceptor (resolves tenant context)
│   ├── interfaces/             # e.g., IDatabaseAdapter contract
│   └── exceptions/             # Global error filters
│
├── modules/                    # Core Application Modules
│   │
│   ├── control-plane/          # GOVERNANCE (No tenant data touches here)
│   │   ├── tenant/             # Tenant lifecycle and DB credential management
│   │   ├── metadata/           # CRUD for schema definitions (The Master Documents)
│   │   └── iam/                # Auth & Policy Engine (RBAC/ABAC definitions)
│   │
│   ├── data-plane/             # EXECUTION (Stateless runtime)
│   │   ├── dynamic-api/        # The single DynamicController & DynamicService
│   │   ├── validation/         # Zod/AJV dynamic schema compilers
│   │   └── transformation/     # Payload normalizers
│   │
│   ├── engines/                # THE ADAPTERS (Database agnostic translators)
│   │   ├── core/               # DatabaseProvider Factory
│   │   ├── sql/                # Knex.js implementation
│   │   └── nosql/              # MongoDB Native implementation
│   │
│   └── runtime/                # CUSTOM LOGIC (User-defined operations)
│       ├── hooks/              # isolated-vm sandbox manager
│       └── background-jobs/    # BullMQ async workers
│
└── infrastructure/             # INTERNAL SERVICES
    ├── cache/                  # Redis connection manager
    └── system-db/              # MongoDB/Mongoose connection for the Control Plane
```

---

## 5. System Maturity Stages (Action Plan)

Building an App Factory requires pragmatic, incremental steps. We cannot build the Query DSL, the Hook Sandbox, and the Billing Engine simultaneously.

### Stage 1: Logical Multi-Tenancy & Metadata Foundation

- Setup NestJS with the modular DDD structure.

- Implement the `infrastructure/system-db` using **MongoDB** to store the `TenantMetadata` JSON.
- Implement the `TenantInterceptor` to resolve tenant context from requests.

## Stage 2: The Adapter Pattern & Basic Data Plane

- Define the `IDatabaseAdapter` interface.
- Implement a basic `SqlEngine` (Knex) and `NoSqlEngine` (Mongo).
- Create the `DynamicController` capable of routing basic CRUD operations to the correct injected adapter.

## Stage 3: Dynamic Validation

- Integrate Zod or AJV in the `data-plane/validation` module.
- Ensure that every `POST` or `PATCH` request is validated against the specific schema defined in the tenant's MongoDB metadata document.

## Stage 4: Advanced Query DSL & Relations

- Evolve the `DynamicService` to understand an internal Query DSL (filtering, sorting, pagination).
- Implement translation logic in the Adapters to convert the DSL into complex SQL Joins or Mongo Aggregations.

## Stage 5: Hook Execution & Background Jobs

- Integrate `isolated-vm` to allow users to save and execute custom JS code securely.
- Implement **BullMQ** to move long-running tasks (like webhooks or email dispatching triggered by hooks) out of the main request cycle.

## Stage 6: Enterprise API Gateway & Observability

- Build the multi-tenant API Gateway using NestJS Guards/Interceptors to enforce **Rate Limiting, CORS, and WAF** rules dynamically based on the Master Document.
- Implement tenant-scoped metrics and distributed tracing to monitor **p95 latency** and error rates per individual tenant.

### Stage 7: Billing Engine & Schema Evolution

- Refactor the Data Plane to emit asynchronous **Usage Events** (reads, writes, hook CPU time) to the Control Plane.
- Build the Billing Aggregator in the Control Plane to compute tier limits and invoices.
- Implement **Metadata Versioning** ( `version: 1 -> version: 2` ) to allow safe, rollback-ready schema evolution without zero-downtime migrations.

## 6. Research & Validation: The Power of Metadata

According to industry research on building metadata-driven architectures (e.g., Building a Metadata-Driven Data Architecture), metadata acts as the central "compass" that transforms raw data into governable, navigable assets. While traditional data engineering applies this to Data Lakes, our `mini-baas` platform applies these exact same principles to **Application Programming Interfaces (APIs)**.

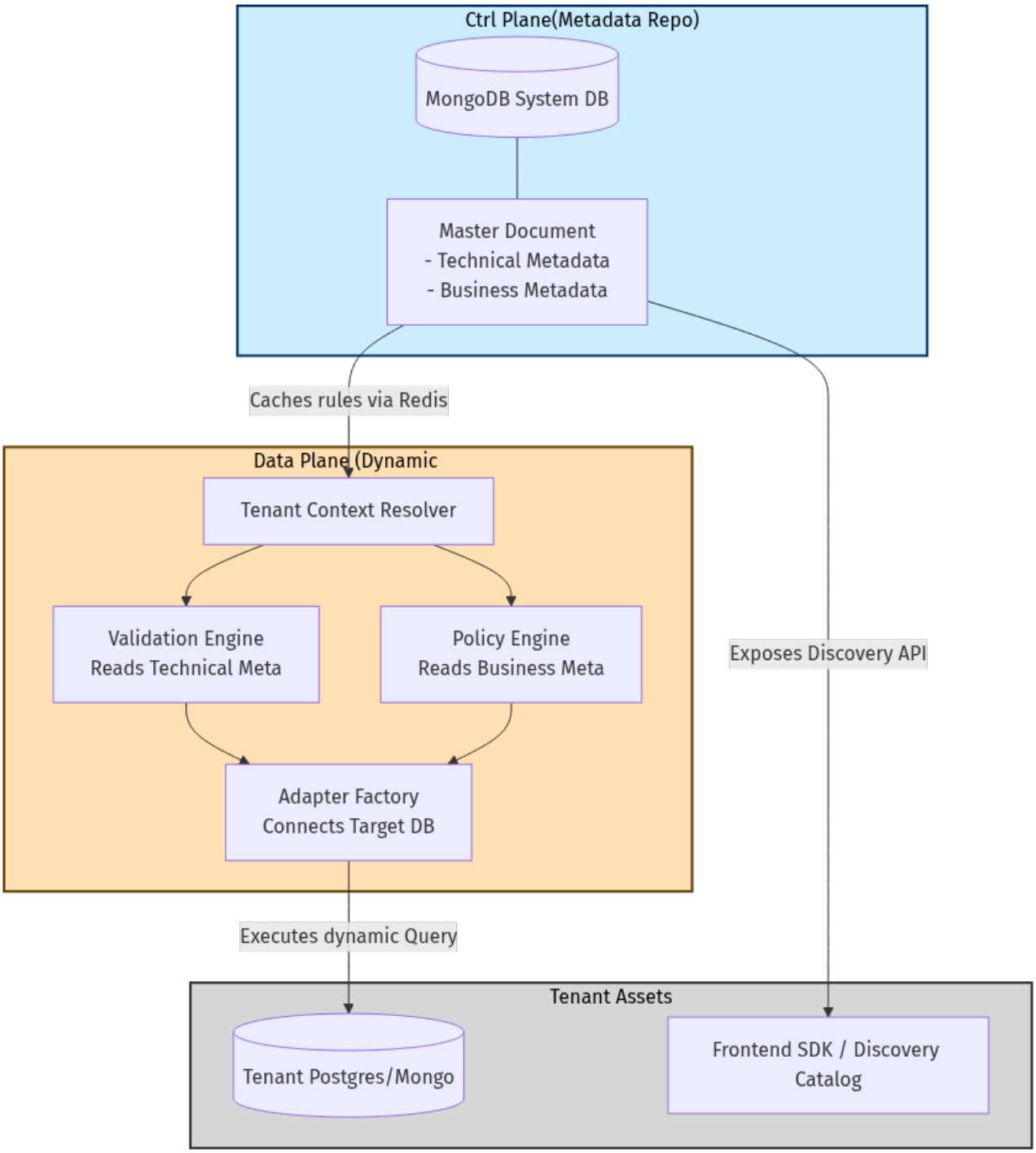
### Alignment with our App Factory

The research highlights three types of metadata (Technical, Operational, Business) and three building blocks (Repositories, Catalogs, Lineage). Here is how they perfectly map to our Control Plane / Data Plane architecture:

1. **Metadata Repositories = Our System DB (MongoDB):** The article defines this as the "heart of a metadata-driven architecture" acting as the single source of truth. In `mini-baas`, our MongoDB Control Plane serves this exact purpose. It stores the `TenantMetadata` JSON (the Master Document) instead of hardcoding models in the backend source code.
2. **Technical & Business Metadata = The Schema & Permissions Objects:** The research notes technical metadata includes "database schemas and data types", while business metadata covers "access controls". Our Master Document explicitly handles both via the `"schema"` property (used by our Adapter Factory and Zod validators) and the `"permissions"` property (used by our IAM/Policy engine).
3. **Data Catalogs = Our Frontend Discovery API:** Catalogs make metadata "searchable and discoverable". In our ecosystem, this translates to the `/discovery` endpoint, which reads the Control Plane metadata and tells the universal Frontend SDK what entities exist, enabling automatic UI generation without hardcoded API routes.
4. **Governance & Lineage = Hooks & Background Jobs:** The article stresses documenting data flows for compliance. Our `isolated-vm` Hooks and telemetry/billing events act as our operational metadata generators, tracking when records are manipulated and enforcing business rules globally across the Data Plane.

## Architectural Flow Diagram

The following diagram illustrates how the building blocks from the research are orchestrated within our specific BaaS infrastructure:



# 7. Enterprise SaaS Capabilities: Scalability, Observability & Evolution

To elevate `mini-baas` from a dynamic API engine to a production-ready SaaS platform, we align our architecture with industry standards for multi-tenancy, observability, and billing (e.g., concepts discussed by UMA Technology and Coretus Technologies).

## 7.1 Scalability and API Gateway Multitenancy

Before a request even reaches our `DynamicController`, it must pass through an API Gateway layer (implemented via NestJS Guards and Interceptors).

- **Tenant-Aware Caching:** As mentioned in our isolation strategy, data contamination is prevented by strictly namespacing Redis keys ( `tenant:ws_123:query_cache` ).
- **Dynamic CORS & WAF:** Cross-Origin Resource Sharing (CORS) and Web Application Firewall (WAF) rules are not globally hardcoded. They are stored in the Control Plane's Master Document. The gateway applies them per request, ensuring one tenant's compromised frontend doesn't affect others.
- **Rate Limiting:** We enforce API limits based on the tenant's subscription tier, dropping excess requests at the gateway before they consume Data Plane CPU.

## 7.2 Multi-Tenant Observability

When running a platform where every request looks different, generic monitoring is blind. We must monitor *per tenant* to react to bottlenecks.

- **Tenant-Scoped Metrics:** Every log, metric, and distributed trace emitted by the Data Plane automatically injects the `tenantId`.
- **Understanding p95 Latency:** We track the **95th percentile (p95)** latency per tenant. If a tenant's `p95 = 240ms`, it means 95% of their requests complete in under 240ms, and only the slowest 5% exceed it. This allows us to detect if a specific tenant wrote a bad database query that is slowing down their specific Data Plane instance, without setting off global alarms.

## 7.3 Billing & Usage Metering (The Event-Driven Approach)

A mature BaaS doesn't just serve data; it measures it. However, calculating bills inside the API request cycle would destroy performance.

Instead, our Data Plane is strictly an *emitter* of **Usage Events**. Every time a query executes, a file is uploaded, or an `isolated-vm` hook runs, the Data Plane pushes an event to an asynchronous queue (**BullMQ**).

```
// Example of a Usage Event emitted by the Data Plane
{
  "tenantId": "ws_123",
  "eventType": "database_read",
  "unitsConsumed": 15, // e.g., rows returned
  "timestamp": "2026-03-04T17:55:00Z"
}
```

The Control Plane houses the **Billing Aggregator** (Accumulator), which consumes these events asynchronously, calculates tier limits, and generates invoices.

## 7.4 Schema Evolution & Metadata Versioning

Traditional applications handle database schema changes via downtime-inducing SQL migrations ( `ALTER TABLE` ). In our Metadata-Driven architecture, schema evolution is instantaneous and reversible (a concept widely praised in O'Reilly's SaaS architecture literature).

- **Version Tags:** Notice the `version: 1` field in our `TenantMetadata` Master Document.
- **Rollbacks:** When a tenant modifies their schema (e.g., adds a required column), we do not overwrite the document. We generate a new Master Document with `version: 2`. If the tenant's frontend breaks, they can instantly rollback the API to `version: 1` by simply flipping the active version pointer in the Control Plane.

This guarantees that metadata changes—the most dangerous operation in a BaaS—are safe, version-controlled, and instantly deployable.